

StayEase Hotel Management System

Software Requirements Specification · IEEE Std 830-1998

Version	1.0
Status	Approved
Prepared by	AgentForge Studio · Agent 1 (Analyzer)
Date	2026-05-08
Architecture	Microservice MERN stack

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope of the Project
- 1.3 Definitions, Acronyms and Abbreviations
- 1.4 References
- 1.5 Overview

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions and Dependencies

3. Specific Requirements

- 3.1 External Interface Requirements
- 3.2 Functional Requirements
- 3.3 Performance Requirements
- 3.4 Security Requirements
- 3.5 Safety Requirements
- 3.6 Capacity Requirements
- 3.7 Availability Requirements
- 3.8 Software System Attributes
- 3.9 Requirement Traceability Matrix

4. Appendix — Diagrams

- Figure A.1 System Context Diagram
- Figure A.2 Use Case Diagram
- Figure A.3 Booking & Payment Sequence
- Figure A.4 Entity Relationship Diagram
- Figure A.5 Deployment Diagram

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) documents the functional and non-functional requirements for StayEase, a multi-property Hotel Management System (HMS).

It is the agreed contract between the development team, QA, and the partner hotel operations group. It captures what the system must do, the constraints under which it operates, and the criteria by which the delivered software will be accepted.

1.2 Scope of the Project

StayEase covers the full reservation lifecycle — discovery, booking, payment, check-in, stay management, and check-out — together with the back-office workflows that hotel staff depend on (room inventory, rate plans, housekeeping, billing, and revenue reporting).

The system has three primary user roles: the Guest who reserves and pays online, the Receptionist who handles walk-ins and supports guests, and the Hotel Manager who oversees inventory, pricing, and reports.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
HMS	Hotel Management System
MERN	MongoDB · Express · React · Node.js technology stack
DBMS	Database Management System
JWT	JSON Web Token — used for stateless authentication
OAuth2	Open Authorization 2.0 — used for social sign-in
API	Application Programming Interface
REST	Representational State Transfer — HTTP-based API style
SLA	Service Level Agreement
PCI-DSS	Payment Card Industry Data Security Standard
GDPR	General Data Protection Regulation (EU 2016/679)
RTM	Requirement Traceability Matrix
FR	Functional Requirement
NFR	Non-Functional Requirement
ECS	AWS Elastic Container Service
ADR	Average Daily Rate (hotel KPI)
RevPAR	Revenue Per Available Room (hotel KPI)
Folio	Itemised statement of charges for a guest stay
Walk-in	A guest who books at the front desk without prior reservation

1.4 References

- [1] IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- [2] PCI-DSS v4.0, Payment Card Industry Data Security Standard.
- [3] EU 2016/679 (GDPR), General Data Protection Regulation.
- [4] OWASP Application Security Verification Standard (ASVS) v4.0.3.
- [5] Stripe API Reference — <https://stripe.com/docs/api>.
- [6] MongoDB 7.0 Manual — <https://www.mongodb.com/docs/manual/>.
- [7] Express.js Guide — <https://expressjs.com/en/guide/routing.html>.
- [8] React 18 Documentation — <https://react.dev/learn>.

1.5 Overview

Section 2 describes the overall product perspective, primary functions, user characteristics, constraints, and assumptions.

Section 3 specifies the external interfaces, the numbered functional requirements (FR1–FR28), the non-functional requirements grouped by category (performance, security, safety, capacity, availability), the software system attributes, and the Requirement Traceability Matrix.

Section 4 contains the appendix diagrams (Context, Use Case, Sequence, ERD, Deployment) rendered from PlantUML.

2. Overall Description

2.1 Product Perspective

StayEase is a new, self-contained product delivered as a microservice MERN stack. The frontend is a Next.js + React single-page application served via CloudFront. The backend consists of an Express API gateway and seven microservices running on AWS ECS Fargate. Persistence is on a MongoDB Atlas managed cluster.

External integrations are limited to Stripe (payments), Postmark (transactional email), and Twilio (SMS). See §4 for the diagrams.

2.2 Product Functions

- Search and book rooms with date, capacity, price, and amenity filters.
- Manage reservations (modify, cancel, no-show handling).
- Take secure online payments and issue refunds.
- Send confirmation, reminder, and cancellation messages by email and SMS.
- Manage room inventory, rate plans, and seasonal pricing.
- Coordinate housekeeping status and turnover times.
- Produce occupancy, ADR, and RevPAR reports for hotel managers.
- Provide role-based access for guests, staff, and platform administrators.

2.3 User Characteristics

Role	Profile and responsibilities
Guest	Non-technical end-user. Browses rooms, makes and manages reservations, pays online.

Receptionist	Hotel staff. Handles walk-ins, takes payments at the front desk, answers guest enquiries.
Housekeeping	Hotel staff. Marks rooms clean / dirty / out-of-order; flags maintenance issues.
Hotel Manager	Power user. Manages inventory, rate plans, cancellation policies; consumes reports.
Platform Admin	Internal user. Manages tenants, integrations, audit trails.

2.4 Constraints

- C-1 Architecture shall be a microservice MERN stack (Mongo + Express + React + Node).
- C-2 All payment flows shall be PCI-DSS SAQ-A compliant; card data shall never traverse the StayEase backend.
- C-3 All guest data shall be processed in line with GDPR; right-to-erasure shall be supported.
- C-4 The system shall be horizontally scalable; no single-instance state.
- C-5 Deployment shall be reproducible from infrastructure-as-code only.

2.5 Assumptions and Dependencies

- A-1 Guests have access to a modern web browser and an active email address.
- A-2 The hotel maintains a contracted Stripe account.
- A-3 Property data (rooms, photos, descriptions) is provided by the hotel during on-boarding.
- D-1 Stripe API availability is required for online payments.
- D-2 Postmark / Twilio availability is required for transactional notifications.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Guest portal shall be a responsive Next.js SPA supporting screens from 360 px (mobile) to 1920 px (desktop). The Operator console shall be a desktop-first Responsive. Both shall conform to WCAG 2.1 Level AA.

3.1.2 Software Interfaces

Layer	Implementation
Frontend	React 18 + Next.js 15 + Tailwind CSS 3
API gateway	Express 4 (Node.js 20 LTS)
Microservices	Node.js 20 + Express + Mongoose
Database	MongoDB 7 (Atlas managed cluster)
Queue	Redis 7 — outbox + retry queue for notifications
Auth	JWT (jsonwebtoken) + Argon2id (argon2)
Payments	Stripe Checkout + Webhooks
Email / SMS	Postmark (email) + Twilio Programmable SMS

Containerisation	Docker 24 + docker-compose v2
Cloud	AWS ECS Fargate + MongoDB Atlas + CloudFront

3.1.3 Hardware Interfaces

Side	Resolution	Processor	RAM	Disk
Server	—	AWS Fargate vCPU 0.5–4	1–8 GB	Ephemeral 20 GB (data on Atlas)
Client	"e 1 2 8 0 × 7 2 0	Any modern CPU	"e 2 G B	—

3.1.4 Communication Interfaces

- All client–server traffic shall use HTTPS with TLS 1.2 or above.
- Inter-service traffic inside the cluster shall use mTLS where supported.
- Public webhooks (Stripe, Twilio) shall be authenticated via signed-payload verification.
- WebSocket channels shall be used only for non-critical UI updates (typing, presence).

3.2 Functional Requirements

3.2 Customer Registration

ID	Requirement
FR1	The Customer should be able to register with their personal details (full name, email, password, phone, date of birth).
FR2	The system shall persist customer details into the users collection (Mongo) keyed by email.
FR3	The system shall send an email verification link valid for 24 hours after sign-up.
FR4	The system shall enforce password complexity ("e 8 chars, at least symbol).

3.2 Authentication

ID	Requirement
FR5	The system shall verify the customer email and password against the users collection at login.
FR6	On successful login, the system shall issue a JWT access token (15 min) and a refresh token (7 days).
FR7	The system shall support OAuth2 sign-in with Google and Facebook providers.
FR8	The system shall lock accounts after 5 consecutive failed login attempts within 15 minutes.

3.2 Room Search & Reservation

ID	Requirement
FR9	The system shall let a guest filter rooms by check-in date, check-out date, occupancy, and price range.

FR10	The system shall display real-time availability calculated from confirmed and pending reservations.
FR11	The system shall allow a guest to hold a quote for 10 minutes before payment is required.
FR12	The system shall enforce check-out dates strictly after check-in dates.
FR13	The system shall record confirmed reservations in the bookings collection with status `confirmed`.
FR14	The system shall send email and SMS confirmation within 60 seconds of payment success.
FR15	The system shall allow a guest to cancel a reservation up to the property's cancellation cut-off.

3.2 Payment Management

ID	Requirement
FR16	The system shall create a Stripe Checkout session and redirect the guest to the Stripe-hosted page.
FR17	The system shall verify Stripe webhook signatures on every event and reject any invalid signature.
FR18	The system shall be idempotent on webhook delivery — re-processing the same event id shall not double-confirm a booking.
FR19	The system shall record refunds against the original payment and reflect them in the reservation timeline within 5 minutes.

3.2 Receptionist / Front-Desk Access

ID	Requirement
FR20	The system shall allow a receptionist to take a walk-in booking and collect payment in person.
FR21	The system shall allow a receptionist to add, update or cancel any booking.
FR22	The system shall provide a customer service desk inbox for the receptionist to answer guest enquiries.

3.2 Manager Access

ID	Requirement
FR23	The system shall generate occupancy, ADR, and RevPAR reports for any user-selected date range.
FR24	The system shall give the manager full create/update/delete access to rooms, rate plans, and policies.
FR25	The system shall maintain an immutable audit log of every privileged action.
FR26	The system shall export reports as both CSV and PDF.

3.2 Housekeeping

ID	Requirement
FR27	Housekeeping users shall be able to update a room's status to clean / dirty / out-of-order.

FR28 Status changes shall propagate to the availability service in under 5 seconds.

3.3 Performance Requirements

ID	Requirement
NF1	Database writes shall complete within 2 seconds at the 95th percentile under normal load.
NF2	Read query results shall return within 5 seconds at the 95th percentile under normal load.
NF3	UI page load shall not exceed 2 seconds on a 3G connection profile.
NF4	Login validation shall complete within 3 seconds.
NF5	Booking-creation endpoint shall sustain at least 50 reservations per second per region without lost updates.

3.4 Security Requirements

ID	Requirement
NF6	All external communication between the data server and clients shall be encrypted with TLS 1.2 or above.
NF7	All personal and payment data shall be stored encrypted at rest using AES-256.
NF8	Payment processing shall use HTTPS and shall comply with PCI-DSS SAQ-A scope (no card data on the StayEase backend).
NF9	Passwords shall be stored as Argon2id hashes with a per-user salt.
NF10	All authentication endpoints shall be rate-limited to a maximum of 10 attempts per 15-minute window per IP.

3.5 Safety Requirements

ID	Requirement
NF11	Database backups shall be taken every hour with 7-day point-in-time recovery.
NF12	Under failure, the system shall be restored to normal operation within 15 minutes.
NF13	No single user action shall cause irreversible data loss without an explicit confirmation step.

3.6 Capacity Requirements

ID	Requirement
NF14	The system shall scale to support at least 100,000 registered customers per property group.
NF15	The system shall handle at least 200 concurrent transactions during peak hours without degradation.

3.7 Availability Requirements

ID	Requirement
NF16	The system shall maintain 99.5% monthly uptime SLA.

NF17	Reports shall be regenerated automatically every 24 hours and on-demand from the manager dashboard.
------	---

3.8 Software System Attributes

Correctness

The system shall satisfy the day-to-day hotel-management operations precisely so that end-user objectives are fulfilled without manual reconciliation.

Efficiency

Hardware and runtime resources shall be allocated such that each tenant's workload completes without contention; horizontal scaling shall be possible without code changes.

Flexibility

The microservice architecture shall accommodate new features (loyalty, channel managers, etc.) by adding services rather than modifying existing ones.

Integrity

All write operations shall be transactional or compensated, so that partial failures cannot leave the database in an inconsistent state.

Interoperability

The system shall expose JSON over HTTPS for all integrations and ship an OpenAPI 3.1 specification for every public endpoint.

Maintainability

Each microservice shall ship with "e 80% unit-test coverage, an OpenAPI 3.1 specification, and a documented runbook.

Portability

Services shall run on any Linux x86_64 host capable of executing the project's container images; no provider-specific dependencies in code.

Reliability

The system shall meet a 99.5% monthly uptime SLA and recover from a single-AZ outage within 60 minutes (RTO).

Reusability

Shared validation, authentication, and audit-logging logic shall live in a versioned `packages/shared/` workspace consumed by every service.

Robustness

Each microservice shall enforce input validation and defensive defaults; downstream failures shall be retried with exponential backoff and circuit-breaker fallbacks.

Testability

Each service shall expose a `/health` and `/ready` endpoint and shall ship a Docker Compose harness for end-to-end testing on a developer laptop.

Usability

A non-technical first-time guest shall be able to complete a booking in under four minutes; the

operator console shall conform to WCAG 2.1 Level AA.

3.9 Requirement Traceability Matrix

Requirements	Feature	SRS Ref	Design	Test cases
FR1–FR4	Registration	§3.2.1	auth-service · POST /auth/register	TC-AUTH-01..05
FR5–FR8	Authentication	§3.2.2	auth-service · POST /auth/login + refresh	TC-AUTH-06..12
FR9–FR15	Reservation	§3.2.3	room-service · GET /rooms · POST /bookings	TC-BOOK-01..14
FR16–FR19	Payment	§3.2.4	payment-service · POST /checkout · webhook	TC-PAY-01..09
FR20–FR22	Receptionist Access	§3.2.5	operator-console · /admin/walk-in	TC-REC-01..06
FR23–FR26	Manager Access	§3.2.6	report-service · /reports/*	TC-MGR-01..08
FR27–FR28	Housekeeping	§3.2.7	housekeeping-service · PATCH /rooms/:id	TC-HK-01..04
NF1–NF5	Performance	§3.3	Load test gate · k6	TC-PERF-01..05
NF6–NF10	Security	§3.4	TLS, Argon2id, rate-limit middleware	TC-SEC-01..10
NF11–NF13	Safety / Recovery	§3.5	Atlas PITR, DR runbook	TC-SAF-01..03
NF14–NF15	Capacity	§3.6	ECS auto-scaling 1–6	TC-CAP-01..02

4. Appendix — Diagrams

Figure A.1 System Context Diagram

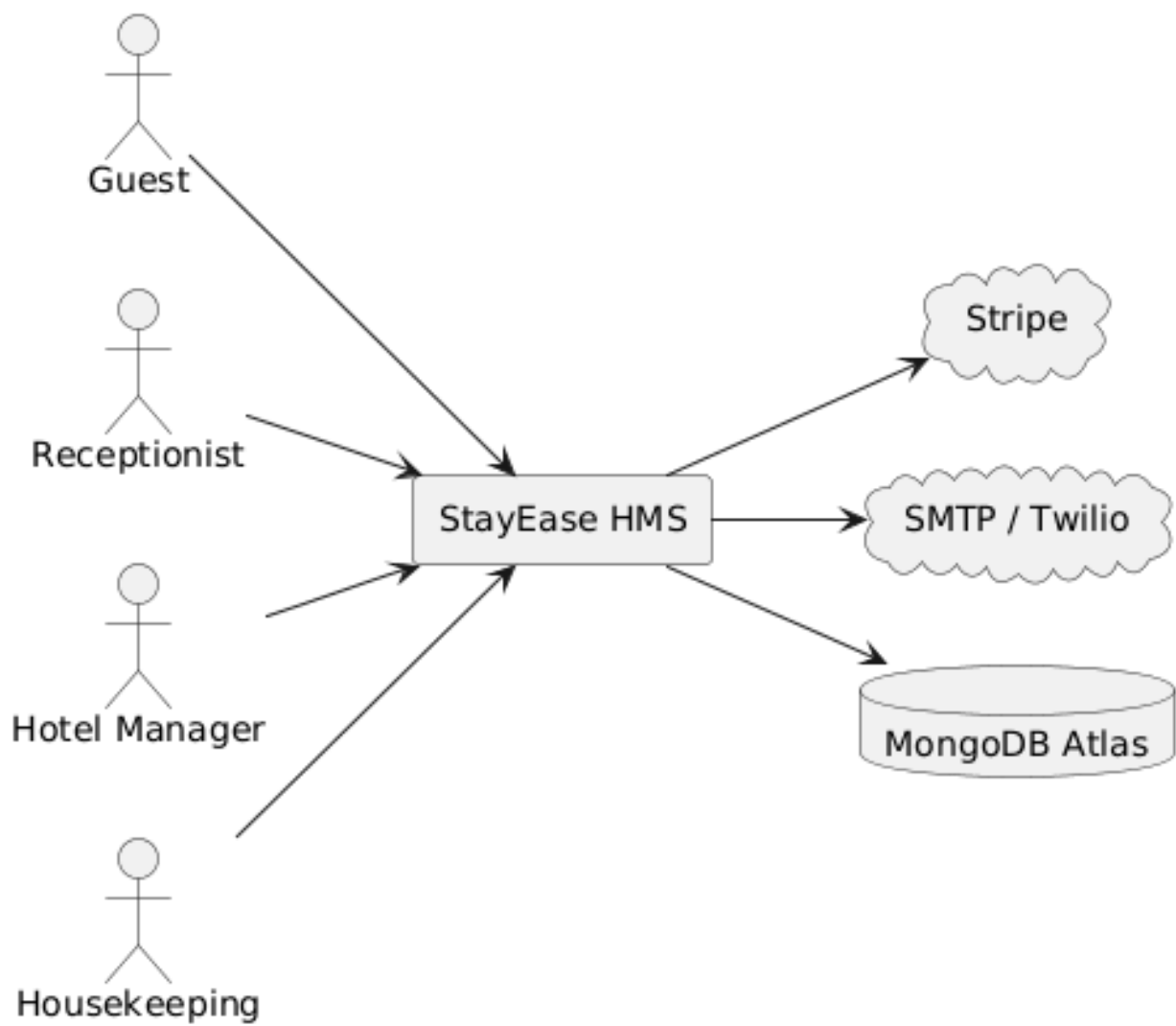


Figure A.1 System Context Diagram

Figure A.2 Use Case Diagram

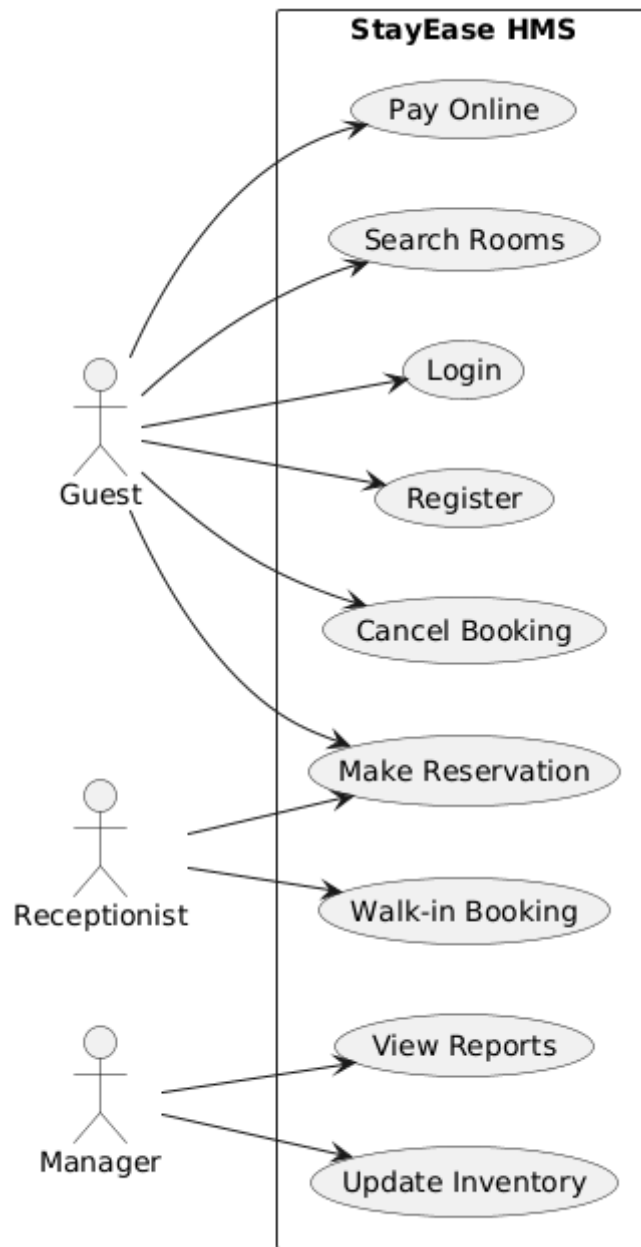


Figure A.2 Use Case Diagram

Figure A.3 Booking & Payment Sequence

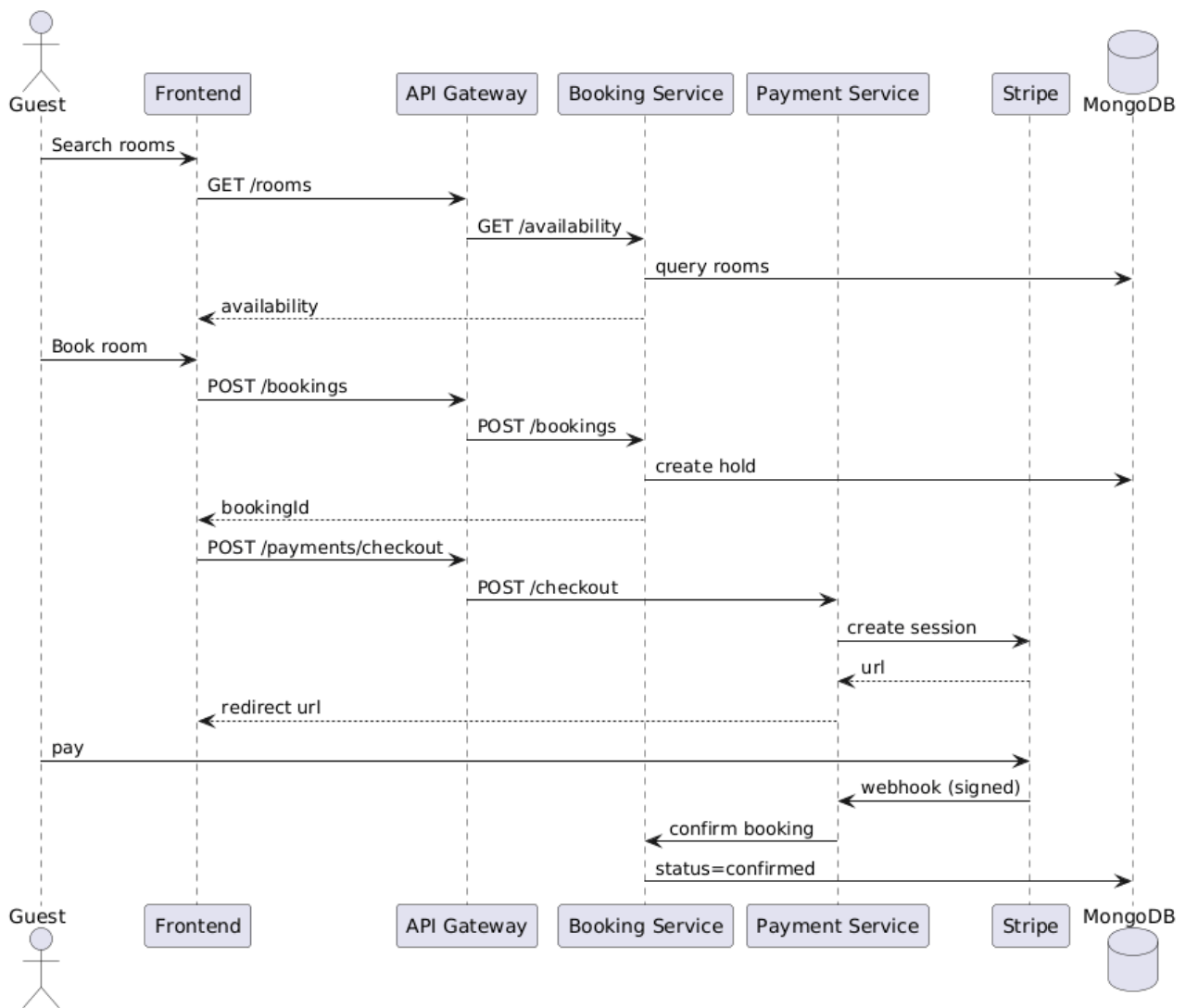


Figure A.3 Booking & Payment Sequence

Figure A.4 Entity Relationship Diagram

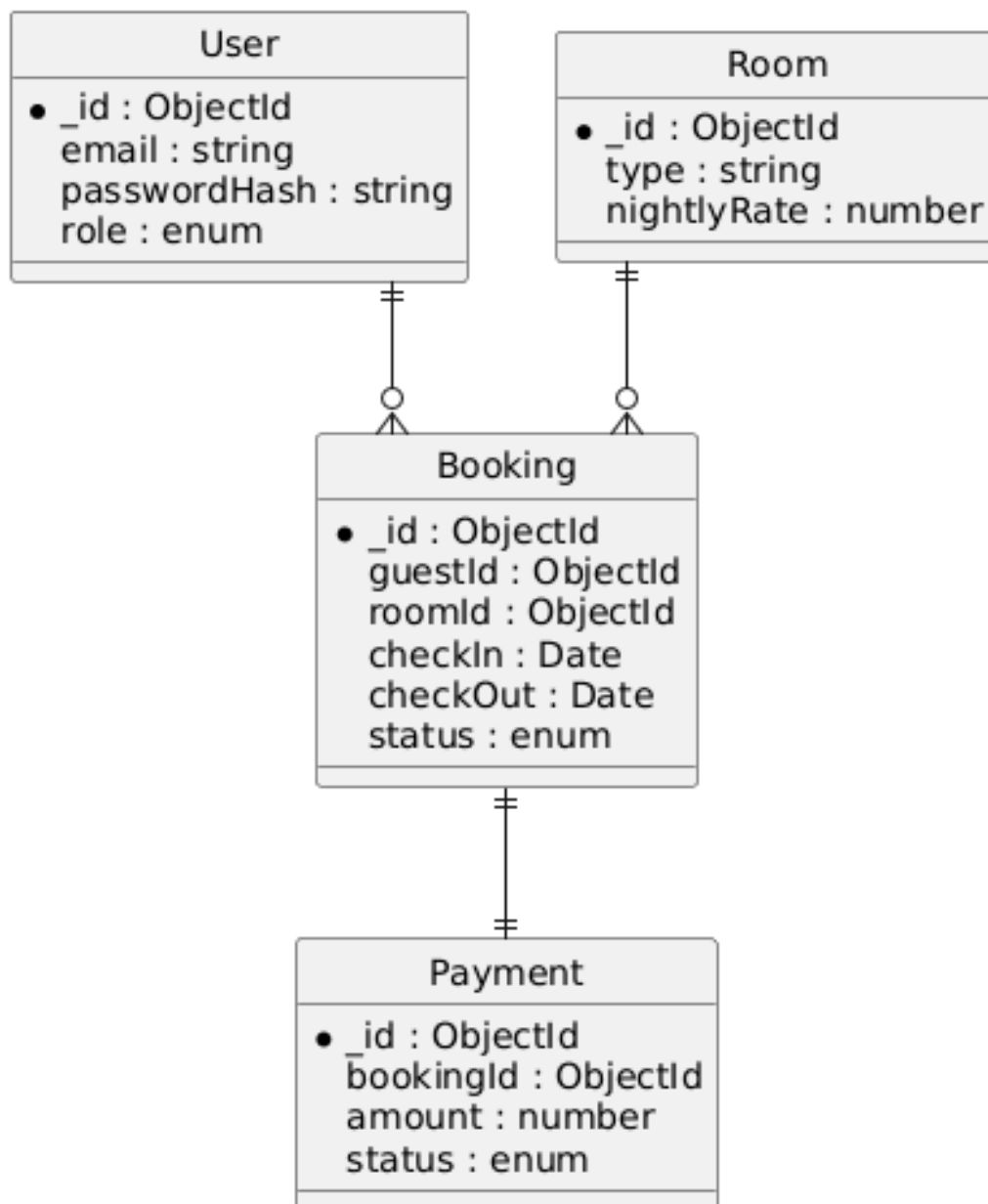


Figure A.4 Entity Relationship Diagram

Figure A.5 Deployment Diagram

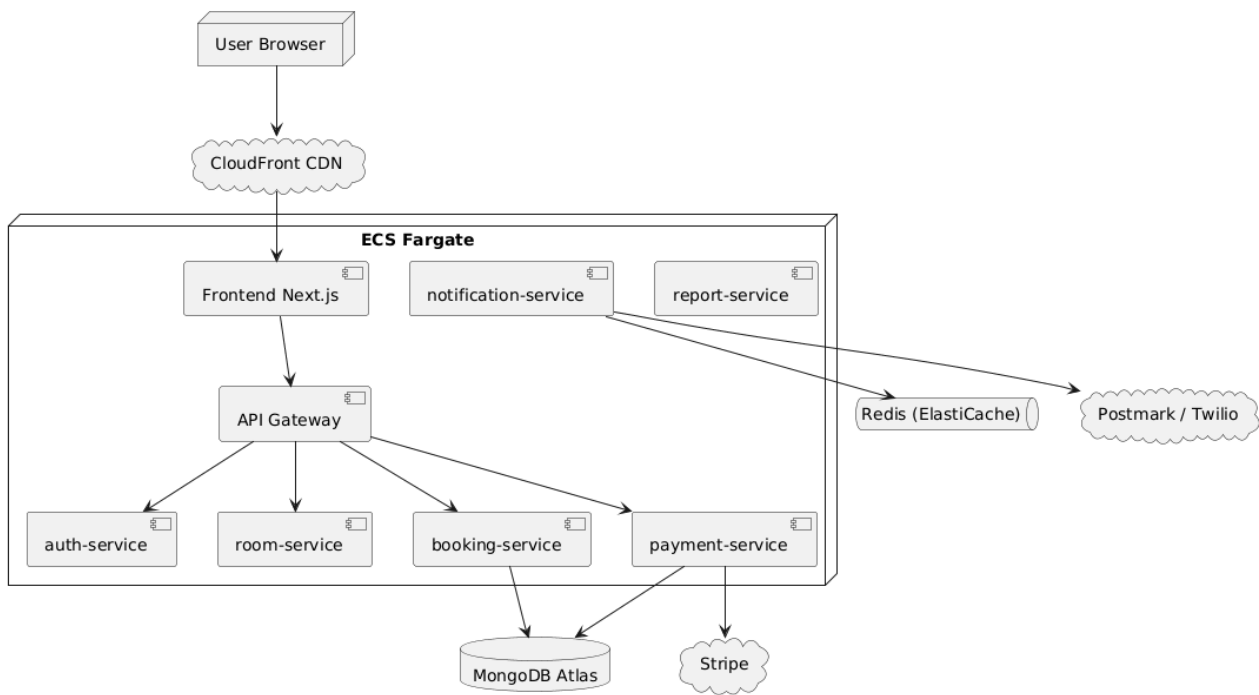


Figure A.5 Deployment Diagram